Informe de ventas en paralelo

Realiza el informe de ventas de 8 tiendas en paralelo: el padre (SalesReport) lanza un hijo ShopSummary por tienda, recoge sus resultados y compara el tiempo uno a uno con el de todos a la vez. Incluye además una caja: dos procesos unidos por una tubería, una caja (CashRegister) que emite tickets y un registro (Ledger) que los suma.

Grupo

Nombre	Qué ha hecho
Adrián De Francisco	SalesReport (Fases 1,2,3,5 lanzando todos a la vez y uno a uno)
Farah Mouaouia Chaarou	CashRegister y Ledger (fase 4, productor y consumidor de la caja en directo)
[Carlos Hurtado Cortes]	[LiveCashRegister(fase 4, unir 2 procesos por una tubería y lanzar los 2 escenarios planteados]

Cómo se ejecuta







Procesos

Clase	Papel	Entrada	Salida	Códigos de salida
`SalesReport`	Padre: lanza un ShopSummary por tienda, recoge sus resultados y escribe el informe	Ninguna	Estándar: una línea por tienda con su PID, el total, las tiendas con problemas y el tiempo. Error: la de los hijos (heredada)	0: bien
`ShopSummary`	Hijo: suma las ventas de una tienda	Argumento: número de tienda, de 1 a 8	Estándar: unidades;céntimos. Error: el mensaje si algo va mal	0: bien; 1: falta el argumento, no es un número o la tienda no existe

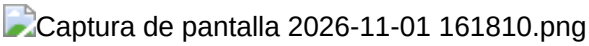
Clase	Papel	Entrada	Salida	Códigos de salida
`LiveCashRegister`	Padre: une CashRegister y Ledger con una tubería y ejecuta los dos escenarios	Ninguna	Estándar: el título de cada escenario y el total del Ledger (heredada). Error: las trazas de los dos hijos (heredada)	0: bien
`CashRegister`	Hijo, productor: emite tickets	Argumentos: número de tickets y pausa en ms	Estándar: un ticket por línea, número;céntimos. Error: hora de cada ticket	0: bien; 1: argumentos que faltan o no son números
`Ledger`	Hijo, consumidor: suma los tickets	Argumento: pausa en ms. Entrada estándar: los tickets	Estándar: número de tickets y total en euros. Error: hora de cada lectura	0: bien; 1: argumento que falta o no es un número

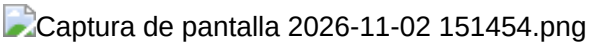
- SalesReport lanza 8 ShopSummary, les pasa la tienda por argumento y lee el resultado de cada uno por su salida estándar; los mensajes de error de los hijos salen en la consola del padre. LiveCashRegister lanza CashRegister y Ledger con startPipeline(): la salida estándar de la caja es la entrada estándar del registro, y espera a los dos.

Pruebas realizadas

Prueba	Resultado esperado	Resultado obtenido
Informe de las 8 tiendas (`SHOPS = 8`)	8 líneas, una por tienda, con su PID. El total que da el enunciado. Tiendas con problemas: 0	Se escriben las 8 líneas, una por tienda, cada una con su PID, sus unidades y su importe. Total: 1.200.014.349 unidades y 3.381.671.938,70 €. Tiendas con problemas: 0
Una tienda que no existe (`SHOPS = 9`)	La tienda 9 falla con un código de salida distinto de 0 y se escribe su error. Tiendas con problemas: 1. El total es el mismo que con 8 tiendas	El hijo escribe "No existe la tienda 9: hay de la 1 a la 8" y termina con código 1. El padre escribe "Tienda 9 (PID 12793): ha fallado con código 1". Tiendas con problemas: 1. Total: 1.200.014.349 unidades y 3.381.671.938,70 €, la suma de las tiendas 1 a 8. Tiempo: 1341 ms
Un hijo que no responde a tiempo (`TIMEOUT = 0`)	Todos los hijos se terminan por tiempo. Tiendas con problemas: 8. Total: 0	Las 8 tiendas escriben "sin respuesta en 0 s; terminada". Total: 0 unidades y 0,00 €. Tiendas con problemas: 8. Tiempo: 631 ms

Prueba	Resultado esperado	Resultado obtenido
Uno a uno frente a todos a la vez	El mismo total en los dos. El tiempo de todos a la vez es menor	El total es el mismo en los dos modos: 1.200.014.349 unidades y 3.381.671.938,70 €. Uno a uno tarda 3921 ms y todos a la vez 1033 ms, unas 3,8 veces menos, porque los 8 hijos trabajan en paralelo en vez de esperarse unos a otros


Captura de pantalla 2026-11-01 161810.png


Captura de pantalla 2026-11-02 151454.png

Tiempos

Tiempos del informe

Los informes de las 8 tiendas (uno a uno y todos a la vez), con sus tiempos y el número de núcleos del equipo, se muestran en las capturas de pantalla adjuntas.

¿Por qué no va 8 veces más rápido?

Porque solo pueden ejecutarse de verdad a la vez tantos procesos como núcleos tiene el equipo, y cada hijo tiene que arrancar su propia JVM. Además, el padre lanza los hijos y lee sus resultados uno tras otro, y esa parte no se paraleliza. Por eso la mejora real queda por debajo del máximo teórico de 8.

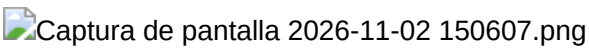
Caja en directo

Quién espera en cada escenario y por qué

Escenario 1 (caja lenta, registro rápido): espera el registro, porque la tubería está vacía y se queda bloqueado en `readLine()` hasta que llega el siguiente ticket. El ritmo lo marca la caja, con un ticket cada 500 ms.

Escenario 2 (caja rápida, registro lento): la caja escribe los 10 tickets en unos 30 ms porque caben en el búfer de la tubería, y el registro los lee a su ritmo (~300 ms cada uno), terminando unos 3 segundos después. Los tickets esperan en la tubería, y la caja solo se bloquearía si se llenara.

En ambos casos la tubería sincroniza los procesos y todo va al ritmo del más lento. Que alguna traza salga desordenada por un milisegundo es normal: son dos procesos con salidas de error independientes.


Captura de pantalla 2026-11-02 150607.png

Depuración


Captura de pantalla 2026-11-01 162302.png


Captura de pantalla 2026-11-02 153526.png


Captura de pantalla 2026-11-02 153629.png

Se ha puesto un punto de interrupción en la línea 53 de `SalesReport`, justo antes de `Process child = launch(shop);`, y se ha ejecutado en modo depuración con `allAtOnce = true` (todos a la vez). El depurador se detiene en el padre, en `report:53`, llamado desde `main:32`.

Lo que se comprueba con la captura:

- **`allAtOnce = true`**: el padre ha entrado en la rama de todos a la vez, donde cada hijo se guarda en la lista `children` y no se espera a que termine antes de lanzar el siguiente.
- **`children` con `size = 0` y `shop = 1`**: aún no se ha lanzado ningún hijo. Si se avanza paso a paso (*Step Over*), la lista crece (1, 2, ... 8) antes de que el padre empiece a esperar a ninguno. Eso confirma que los 8 hijos se lanzan antes de recoger sus resultados, y por eso el tiempo total es menor que uno a uno.
- **`start`**: guarda la hora de inicio en milisegundos, con la que luego se calcula el tiempo del informe.

Más adelante, con los hijos ya terminados, se puede evaluar `child.exitValue()` en la ventana *Evaluate expression* para ver el código de salida de cada tienda (0 si ha ido bien, 1 si ha fallado, como la tienda 9).

Error encontrado depurando

Al cambiar el número de tiendas (`SHOPS = 9`), el informe mostraba una tienda con problemas. Se depuró para ver cuál era el error y se comprobó que la tienda 9 no existe: el hijo escribía "No existe la tienda 9: hay de la 1 a la 8" y terminaba con código 1. No era un fallo del padre, que detectó bien el código de salida distinto de 0 y lo contó como tienda con problemas, así que no hizo falta cambiar el código.

También comprobamos el número de tiendas correcto para ver la ejecuión con código igual a 0.